

Guessing “Which One”

A yes/no answer without a cliff. A score that measures surprise. One example at a time. Then three neurons that share out a vote. By the end, every ingredient of a real network is on the table.

Inside: the sigmoid, a dimmer instead of a switch · surprise as a loss (cross-entropy) · the slope $p - y$, derived in full · stochastic and mini-batch gradient descent · 569 tumours: harmful or harmless? (UCI Breast Cancer Wisconsin) · softmax and one-hot labels · 150 irises: which of three? (UCI Iris) · a classifier in twenty-five lines



$$dL/dz = p - y$$

Where we are in the story

LAST TIME

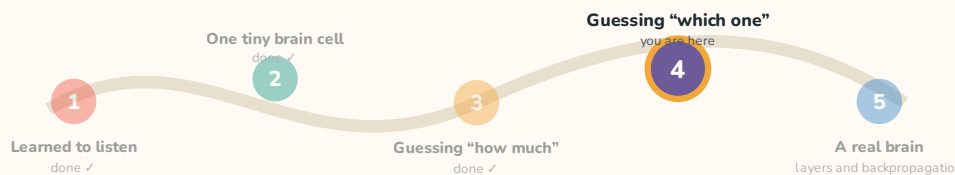
We took the cliff off the perceptron and one neuron could say a **number**. We scored it with the mean squared error, found that the loss is a **bowl**, worked out the slope of the bowl by nudging a knob — $dL/dw = (2/N) \cdot \sum e_i x_i$ — and walked downhill with **gradient descent**, first by hand on four cars and then on 392 real ones.

Two ideas from Book 3 come back on every page here: *error × input* is the gradient, and the **chain rule** is just multiplying the links.

THIS TIME

Back to yes/no questions — but done properly. Instead of a cliff we **squash** the total gently into a number between 0 and 1 and read it as “how sure”. Instead of squared error we score by **surprise**. The gradient turns out to be the simplest formula in the whole course: $p - y$, times the input.

We train one example at a time (**stochastic** gradient descent) on real medical data, then line up three neurons and let **softmax** share a vote between three flower species, trained in small **mini-batches**. After this book, Book 5 only has to stack what you already know.



What you need

Everything from Book 3, plus two small facts about logarithms that Chapter 2 explains as it goes. The derivations use the same nudge-a-knob idea as before; the only new step is that the chain now has three links instead of two. The companion notebook ([Book4_Classification_From_Scratch.ipynb](#)) needs no downloads — both datasets come with scikit-learn.

The boxes you will meet

Story · Big idea · Think of it like... · Stop and think · Our answer · Try it yourself · Careful · Spark · Words to know · numbered derivation boxes, every line shown.

1

Yes or no — gently

The perceptron shouted 0 or 1. The regression neuron said any number. We want something in between: a number between 0 and 1 that means “how sure”.

STORY

Meera, the medical student from Book 2, comes back with a real problem. When a lump is found in a breast, a doctor draws a few cells with a fine needle and looks at them under a microscope. Thirty measurements are made of the cell nuclei — average radius, texture, how jagged the edges are, and so on. The question is the one that matters: **is this tumour malignant (harmful) or benign?**

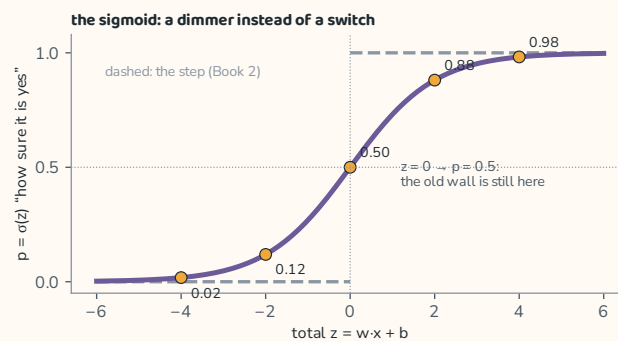
“Yes or no,” says Asha. “That’s Book 2.” Meera shakes her head. “A doctor never wants a bare yes or no. She wants to know *how sure*. A 51% guess and a 99% guess lead to very different conversations with the patient.”

THE SIGMOID

$$z = \mathbf{w} \cdot \mathbf{x} + b, \quad p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$e \approx 2.718$; e^{-z} just means “e to the power minus z”. For many patients at once, $\mathbf{p} = \sigma(\mathbf{X}\mathbf{w} + b)$: the squash is applied to every entry of the column.

Feed it any total z and out comes a number strictly between 0 and 1. A very negative z gives p near 0 (“almost surely no”); z = 0 gives exactly 0.5 (“no idea”); a very positive z gives p near 1 (“almost surely yes”). The shape is an S, smooth everywhere, with no cliff.



The dimmer. The step (dashed) jumps from 0 to 1 at $z = 0$. The sigmoid glides. Notice it still crosses 0.5 at exactly $z = 0$ — the wall from Book 2 is still there; the sigmoid only tells you how far you are from it.

z	-4	-2	-1	0	1	2	4
$p = \sigma(z)$	0.018	0.119	0.269	0.500	0.731	0.881	0.982

Two things worth memorising from the table: $\sigma(0) = 0.5$, and $\sigma(-z) = 1 - \sigma(z)$ (the curve is symmetric: 0.269 and 0.731 add to 1).

Three neurons, one difference

Every neuron in this course computes the same total, $z = \mathbf{w} \cdot \mathbf{x} + b$. They differ only in what they do with it at the end:

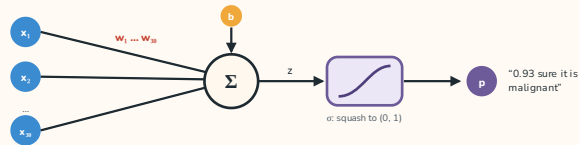
Neuron	Last step	Output	Answers
Perceptron (Book 2)	a cliff: 1 if $z \geq 0$, else 0	exactly 0 or 1	yes / no — but never “almost”
Regression neuron (Book 3)	nothing: $\hat{y} = z$	any number	how much?
Sigmoid neuron (this book)	a gentle squash: $p = \sigma(z)$	a number between 0 and 1	how sure is it yes?

THINK OF IT LIKE... A DIMMER INSTEAD OF A LIGHT SWITCH

The perceptron was a light switch: off or on, nothing between. The sigmoid is a dimmer. Turn the knob (z) up and the light brightens smoothly from dark to full. The brightness is how sure the neuron is. And a dimmer has a wonderful property a switch lacks: a *small* turn *always* makes a *small* change. That is exactly what gradient descent needs — remember, the step function gave it nothing to feel.

Making a decision — and keeping the doubt

When we must answer, we say **yes if $p \geq 0.5$** , otherwise no. Since $p \geq 0.5$ exactly when $z \geq 0$, this is the perceptron's decision, line for line: the same wall, the same two sides. What we have gained is the number itself: $p = 0.51$ and $p = 0.99$ both mean “malignant”, but the doctor sees the difference. We can also move the threshold — “call it malignant if $p \geq 0.2$ ” — to miss fewer cancers at the cost of more false alarms. Chapter 5 does exactly that.



The sigmoid neuron. Thirty measurements in, thirty knobs plus a bias, one total, one squash, one probability out. Compare with the regression neuron of Book 3: only the last box differs.

💡 STOP AND THINK

A neuron with two inputs has $w = (1, 2)$, $b = -3$. For the input $x = (1, 1)$, what is z , what is p , and what is the verdict? And for $x = (3, 2)$?

✓ OUR ANSWER

$x = (1, 1)$: $z = 1 + 2 - 3 = 0$, so $p = 0.5$ — right on the wall, a coin toss. $x = (3, 2)$: $z = 3 + 4 - 3 = 4$, so $p = \sigma(4) = 0.982$ — a confident yes. Same wall as the perceptron; the sigmoid adds the confidence.

2

The score: how surprised were you?

A loss for probabilities. It rewards honest confidence, forgives honest doubt, and punishes arrogance without limit.

The neuron now outputs a probability p . The truth y is 1 (malignant) or 0 (benign). We need a score: one number that says how bad the guess p was, given what actually happened. Book 3's squared error, $(p - y)^2$, is possible — but there is a much better idea, and it comes from asking a simple question.

THINK OF IT LIKE... PAYING A WEATHER FORECASTER

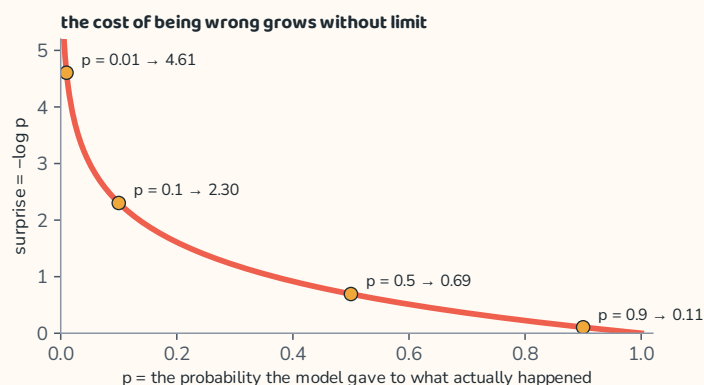
A forecaster says "90% chance of rain". It rains. She was nearly sure, so she gets a tiny penalty. Another says "50%". It rains. He hedged, so a middling penalty. A third says "1% chance of rain". It pours. He was confidently wrong, and he is fined heavily. Now the rule: **the penalty is the surprise you feel when the truth arrives, given what you were told**. Confident and right: no surprise. Confident and wrong: enormous surprise.

SURPRISE

$$\text{surprise} = -\log(p_{\text{truth}})$$

where p_{truth} is the probability the model gave to what actually happened

"log" is the natural logarithm. You need only two facts about it. **log 1 = 0**, so a perfect prediction costs nothing. And **log of a number below 1 is negative**, getting more negative the smaller the number — so $-\log$ is positive, and grows without limit as p shrinks to 0. Every halving of p adds the same 0.69 to the surprise.



Surprise = $-\log p$. Being 90% sure and right costs 0.11. Hedging at 50% costs 0.69 no matter what happens. Being 99% sure of the wrong thing costs 4.61 — forty times more than the honest hedge. The curve has no top: arrogance is unbounded.

Writing it for both answers

If the truth is yes ($y = 1$), the model gave it probability p , so the surprise is $-\log p$. If the truth is no ($y = 0$), the model gave "no" the probability $1 - p$, so the surprise is $-\log(1 - p)$. One formula that covers both, because y switches one term on and the other off:

$$L = -[y \log p + (1 - y) \log(1 - p)]$$

when $y = 1$ only the first term survives; when $y = 0$ only the second

Average this over all the examples and you have the **binary cross-entropy** — also called the **log loss**. Lower is better; zero is perfect. The grand name hides a small idea: *average surprise*.

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)] \quad \text{with} \quad \mathbf{p} = \sigma(\mathbf{X}\mathbf{w} + b)$$

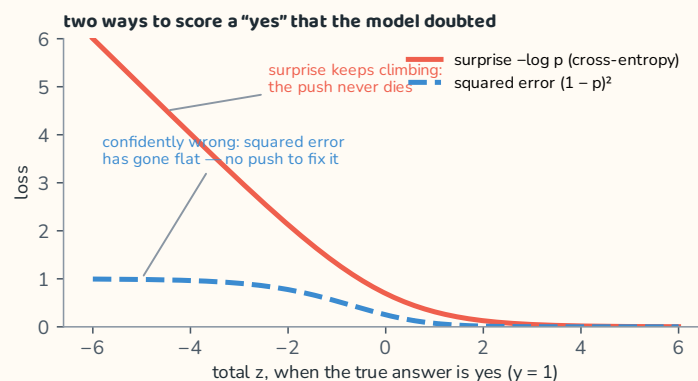
the whole batch: one column of probabilities from one matrix product, one average over its rows — `np.mean(...)` in the notebook

Three patients, by hand

patient	truth y	model's p (malignant)	probability given to the truth	surprise $-\log(\cdot)$
1	1 (malignant)	0.9	0.9	0.105
2	0 (benign)	0.2	$1 - 0.2 = 0.8$	0.223
3	0 (benign)	0.6	$1 - 0.6 = 0.4$	0.916
average surprise (the loss)				0.415

Patient 3 is where the model is losing most: it leaned towards malignant (0.6) for a benign tumour. Compare two other models: one that always says $p = 0.5$ scores $-\log 0.5 = 0.693$ on every patient; a perfect one scores 0. Ours, at 0.415, is between “clueless” and “perfect” — which is what a loss should tell you.

Why not just use the squared error?



Two scores for the same mistake. The truth is yes. As z goes very negative (the model confidently says no), the squared error flattens out at 1 — the slope dies, and gradient descent has nothing to push against. Surprise keeps rising, so the push keeps coming. The more wrong the model, the harder it is corrected.

Two reasons. First, the picture: when a sigmoid neuron is confidently wrong, the squared error goes flat, and a flat loss means a zero gradient — training stalls exactly when it most needs to move. Surprise never goes flat. Second, a quieter reason you will see in the next chapter: with surprise as the loss, the gradient comes out as the same clean “error $\times$ input” shape we have had since Book 2. Squared error through a sigmoid gives a messier, weaker formula.

STOP AND THINK

Two tumours are malignant. Model A says $p = 0.6$ for both. Model B says 0.99 for one and 0.2 for the other. Which has the lower average surprise? Which would you rather have in a clinic?

OUR ANSWER

A: $(0.511 + 0.511)/2 = 0.51$. B: $(0.010 + 1.609)/2 = 0.81$. Model A wins on surprise, even though B was “more right” on one patient — because surprise punishes the confident miss on the other far more than it rewards the confident hit. That is the loss doing its job: in a clinic, one confident miss is worse than two hedges.

SPARK — SURPRISE HAS A UNIT

In 1948 Claude Shannon showed that $-\log_2 p$ measures information in **bits**: a fair coin toss carries exactly 1 bit of surprise, a 1-in-1024 event carries 10 bits. Using the natural log instead of $\log_2$ only changes the unit (to “nats”). So when you train a classifier by minimising cross-entropy, you are literally minimising the number of bits by which the truth surprises your model. The whole of deep learning for classification stands on a 1948 paper about telephone lines.

WORDS TO KNOW

Surprise — $-\log$ of the probability the model gave to what happened. **Binary cross-entropy (log loss)** — average surprise over the examples; our loss. **Natural log** — log base e ; $\log 1 = 0$, $\log \frac{1}{2} = -0.69$. **Calibration** — a model whose 0.9s are right nine times in ten; cross-entropy rewards it.

3

The slope: three links, one beautiful answer

Nudge a knob, and the loss changes through three links: the total, the squash, the surprise. Multiply them and almost everything cancels.

To walk downhill we need $\partial L / \partial w$ and $\partial L / \partial b$ for the sigmoid neuron with the surprise loss. In Book 3 the chain from a knob to the loss had two links: $w \rightarrow e \rightarrow e^2$. Now it has three: $w \rightarrow z \rightarrow p \rightarrow L$. The chain rule says: find the slope of each link, multiply. Let's find the three slopes, one at a time, then multiply — first for one patient with plain numbers, then for a whole batch of patients as matrices.

PLAIN MATHS — one patient, one knob, three links

Link 1: from the knob to the total. $z = w_1 x_1 + w_2 x_2 + \dots + b$. Nudge w_1 by δ and z moves by $x_1 \delta$. So the slope of z with respect to w_1 is the input, and with respect to b it is 1 — exactly as in Book 3; nothing has changed at the front of the neuron.

$$1 \quad \frac{\partial z}{\partial w_1} = x_1 \quad \frac{\partial z}{\partial b} = 1$$

the front of the neuron is a weighted sum, as always

Link 2: from the total to the probability — the slope of the sigmoid. This is the only genuinely new piece of calculus in the course, and it has a famous, tidy answer. Two small facts: the slope of $1/u$ with respect to u is $-1/u^2$, and the slope of e^{-z} with respect to z is $-e^{-z}$. (Check the first with numbers: $1/2 = 0.5$ and $1/2.01 = 0.4975$; a change of -0.0025 for a nudge of 0.01 is a slope of $-0.25 = -1/2^2$. ✓)

$$2 \quad u = 1 + e^{-z}, \quad p = \frac{1}{u}$$

write the sigmoid as "1 over something"

$$3 \quad \frac{\partial p}{\partial z} = \frac{\partial p}{\partial u} \cdot \frac{\partial u}{\partial z} = \left(-\frac{1}{u^2} \right) (-e^{-z}) = \frac{e^{-z}}{u^2}$$

a chain within the chain: how p changes with u , times how u changes with z

$$4 \quad \frac{\partial p}{\partial z} = \frac{1}{u} \cdot \frac{e^{-z}}{u} = p \cdot \frac{e^{-z}}{1 + e^{-z}}$$

split u^2 into $u \cdot u$ and notice that $1/u$ is p

$$5 \quad \frac{e^{-z}}{1 + e^{-z}} = 1 - \frac{1}{1 + e^{-z}} = 1 - p \implies \frac{\partial p}{\partial z} = p(1 - p)$$

the slope of the S-curve is " p times $1 - p$ ": biggest at $p = 0.5$ (slope 0.25), tiny near 0 and near 1. Steep where undecided, flat where sure — and you read it straight off the output.

Link 3: from the probability to the surprise. One more fact: the slope of $\log p$ with respect to p is $1/p$. (Check: $\log 1.01 - \log 1 = 0.00995$ for a nudge of 0.01 , a slope of $0.995 \approx 1/1$. ✓)

$$6 \quad L = -[y \log p + (1 - y) \log(1 - p)] \implies \frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}$$

differentiate each term; the second picks up a minus sign because $1 - p$ falls as p rises

Multiply the links.

$$7 \quad \frac{\partial L}{\partial z} = \underbrace{\frac{\partial L}{\partial p}}_{\text{link 3}} \cdot \underbrace{\frac{\partial p}{\partial z}}_{\text{link 2}} = \left[-\frac{y}{p} + \frac{1 - y}{1 - p} \right] p(1 - p)$$

slope of the loss with respect to the total

$$8 \quad \frac{\partial L}{\partial z} = -y(1 - p) + (1 - y)p$$

multiply through: the first term's p cancels a p ; the second term's $(1 - p)$ cancels a $(1 - p)$

$$9 \quad \frac{\partial L}{\partial z} = -y + yp + p - yp = p - y$$

expand and tidy: the yp terms cancel

$$10 \quad \frac{\partial L}{\partial w_j} = (p - y) x_j \quad \frac{\partial L}{\partial b} = p - y$$

attach link 1 for each knob: **error $\times$ input**

↓ the same, for **every patient at once** — stack them as rows

🔗 MATRIX FORM — N patients, d measurements, one product each

- 1 $\mathbf{z} = \mathbf{X}\mathbf{w} + b$ ($N \times 1$)
every patient's total; row i of $\mathbf{X}$ holds patient i 's d measurements
- 2 $\mathbf{p} = \sigma(\mathbf{z})$ ($N \times 1$)
squash every entry — the sigmoid applied element by element
- 3 $L = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$
average surprise over the batch
- 4 $\nabla_{\mathbf{w}} L = \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$ ($d \times 1$) $\frac{\partial L}{\partial b} = \frac{1}{N} \mathbf{1}^T (\mathbf{p} - \mathbf{y})$
step 10 for every knob and every patient in one line: $\mathbf{X}^T$ times the column of errors adds "error $\times$ input" over the batch — Book 3's $\mathbf{X}^T \mathbf{e}$, with $\mathbf{p} - \mathbf{y}$ as the error and no factor 2

With $N = 1$ this is SGD (Chapter 4): $\mathbf{X}$ is a single row, and $\mathbf{X}^T (\mathbf{p} - \mathbf{y})$ is just $(p - y)$ times that patient's inputs. With $N = 16$ it is a mini-batch. Same line, different number of rows.

🔗 BIG IDEA — IT IS ERROR $\times$ INPUT, AGAIN

Three links, two nasty-looking fractions, and it all collapses to $p - y$: the guess minus the truth. The gradient for each knob is that error times the knob's input. It is Rosenblatt's rule with a real-valued error (Book 2), and Book 3's formula with p in place of $\hat{y}$. The sigmoid's slope $p(1 - p)$ and the surprise's slope $1/p$ were built to cancel each other. That cancellation is the reason cross-entropy and the sigmoid are always used together.

One patient, by hand

Two inputs $\mathbf{x} = (2, 1)$, truth $y = 1$ (malignant), knobs $\mathbf{w} = (0, 0)$, $b = 0$, learning rate $\eta = 1$.

	$z = \mathbf{w} \cdot \mathbf{x} + b$	$p = \sigma(z)$	surprise $-\log p$	$\partial L / \partial z = p - y$	$\nabla_{\mathbf{w}} L = (p - y) \mathbf{x}$	$\partial L / \partial b$	new $\mathbf{w}, b$
before	$0 \cdot 2 + 0 \cdot 1 + 0 = 0$	0.5	0.693	$0.5 - 1 = -0.5$	$-0.5 \cdot (2, 1) = (-1, -0.5)$	-0.5	$\mathbf{w} = (1, 0.5), b = 0.5$
after	$1 \cdot 2 + 0.5 \cdot 1 + 0.5 = 3$	0.953	0.048	-0.047	$(-0.094, -0.047)$	-0.047	(barely moves now)

The same step, as matrices — one patient is a one-row $\mathbf{X}$:

$$z = \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0 = 0, \quad p = \sigma(0) = 0.5, \quad \nabla_{\mathbf{w}} L = \mathbf{X}^T (p - y) = \begin{bmatrix} 2 \\ 1 \end{bmatrix} (0.5 - 1) = \begin{bmatrix} -1 \\ -0.5 \end{bmatrix}, \quad \mathbf{w} \leftarrow \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 1 \cdot \begin{bmatrix} -1 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

One step took the surprise from 0.693 to 0.048. Notice the second-row gradient: once p is close to y , $p - y$ is tiny and the knobs stop moving. The neuron pushes hard when it is wrong and relaxes when it is right — automatically, because the gradient is the error.

🔗 TRY IT YOURSELF

Same start, but the truth is $y = 0$. Compute $p - y$, the gradient, the new knobs ($\eta = 1$), and the new p . Check upside down:

$$p - y = 0.5 - 0 = 0.5; \quad dL/d\mathbf{w} = (1, 0.5); \quad dL/db = 0.5; \quad \mathbf{w} = (-1, -0.5); \quad b = -0.5; \quad z = -3, \quad L = 0.407$$

⚠️ CAREFUL — $p - y$ IS A SLOPE, NOT A MISTAKE COUNT

In Book 2 the error was $+1, -1$ or 0 . Here $p - y$ is any number between -1 and 1 : a 0.51 guess for a malignant tumour gives -0.49 , a 0.99 guess gives -0.01 . The neuron is nudged in proportion to how wrong it was — the very thing the perceptron could not do.

🔗 THINK OF IT LIKE... A CHAIN OF THREE LINKS

Pull the knob, the total moves (link 1: by $\mathbf{x}$). The total moves, the dimmer brightens (link 2: by $p(1 - p)$). The dimmer brightens, the surprise changes (link 3: by $-1/p$). Three links, three slopes, multiply. In Book 5 the chain will run through a whole hidden layer, and the recipe will not change: find each link's slope, multiply along the chain.

4

One at a time: stochastic gradient descent

Book 3 looked at every example before taking a step. Now we look at one, step, look at the next, step. Noisier, cheaper, and — surprisingly — often better.

THINK OF IT LIKE... A CHEF TASTING THE SOUP

A careful chef could pour the whole pot through a machine to measure its saltiness exactly, then add salt, then pour it all through again. That is **batch** gradient descent: perfectly accurate, painfully slow. A real chef takes one spoonful, adjusts a little, stirs, tastes another spoonful, adjusts again. Each spoonful is a noisy sample of the pot — but a hundred spoonfuls get the soup right long before the machine has finished its first pass. That is **stochastic** gradient descent. (“Stochastic” just means “with randomness”: the spoonfuls come from random places in the pot.)

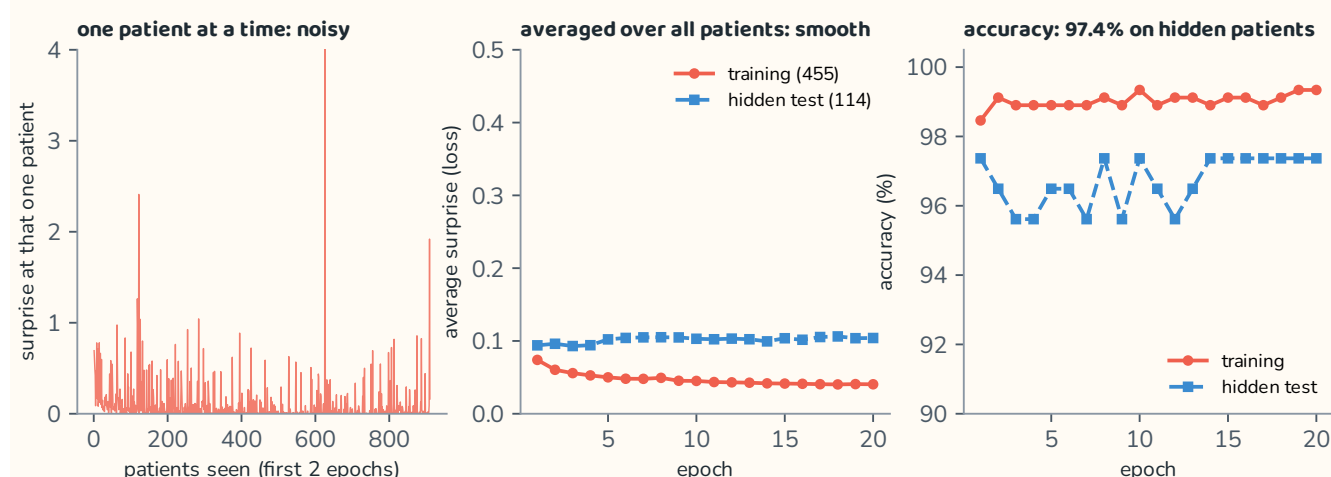
The algorithm

0 • Start	Knobs at 0. Choose a learning rate η — smaller than for batch, because each step is a noisy guess.
1 • Shuffle	Put the training examples in a random order. (Never feed them in a fixed order; the neuron would learn the order.)
2 • One example	Take the next example $(\mathbf{x}_i, y_i)$. Guess: $p = \sigma(\mathbf{w} \cdot \mathbf{x}_i + b)$.
3 • Its gradient	$\nabla_{\mathbf{w}} L = (p - y_i) \mathbf{x}_i$ and $\partial L / \partial b = p - y_i$ — for this <i>one</i> example, no averaging.
4 • Step	$\mathbf{w} \leftarrow \mathbf{w} - \eta (p - y_i) \mathbf{x}_i$, $b \leftarrow b - \eta (p - y_i)$.
5 • Repeat	Back to 2 until every example has been seen once — that is one epoch , and it contains N steps, not 1. Then reshuffle and go again.

Compare with Book 3: there, one epoch was one step using all N examples. Here one epoch is N steps, each using one. Same total amount of arithmetic per epoch — but N moves of the knobs instead of one.

Why noisy steps still get you down the hill

The gradient from one example is a rough estimate of the gradient from all of them. Rough, but not biased: on average, over many examples, it points the right way. So each step is a small move in *roughly* the right direction, and a thousand small roughly-right moves add up to a walk down the hill — a drunken walk, zig-zagging, but downhill. The zig-zags do not cancel the progress; they only make the loss curve jagged when you plot it step by step.



The same training, seen three ways. Left: the surprise at each individual patient during the first two epochs of SGD — wild, because a single patient is a single spoonful. Middle: the average surprise over all patients at the end of each epoch — smooth, and falling. Right: the accuracy on the training patients and on the 114 hidden ones. (This is the real 30-input model of Chapter 5.)

The middle way: mini-batches

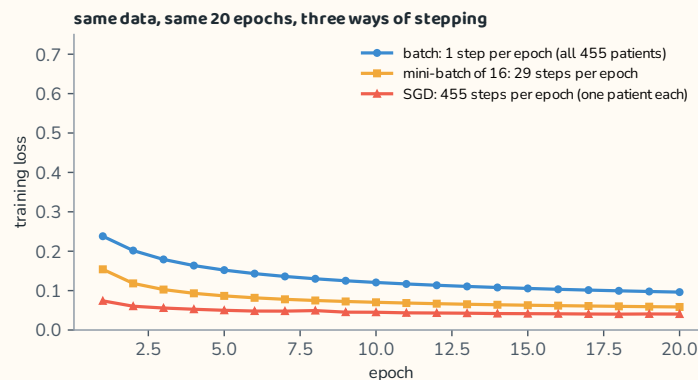
Between “all N” and “just one” sits the choice almost everyone actually uses: a **mini-batch** of a few examples — 16, 32, 64 — averaged into one gradient, then one step. Less noisy than a single example, far cheaper than the whole set. There is also a hardware reason: the chips that run deep learning (GPUs) multiply whole grids of numbers at once, so working on 32 examples costs about the same time as working on 1. Mini-batches get that parallelism for free.

	Batch (Book 3)	Stochastic (SGD)	Mini-batch
examples per step	all N	1	a handful (16–256)
steps per epoch	1	N	N / batch size
gradient quality	exact	very noisy	a little noisy
cost per step	high	tiny	small (and parallel)
typical learning rate	larger	small	medium
used when	data is small	teaching, or streaming data	almost always in practice

One formula, three batch sizes. Take any B rows of the data as $\mathbf{X}_B$ (with labels $\mathbf{y}_B$); the step is always

$$\mathbf{p} = \sigma(\mathbf{X}_B \mathbf{w} + b), \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{B} \mathbf{X}_B^\top (\mathbf{p} - \mathbf{y}_B), \quad b \leftarrow b - \eta \frac{1}{B} \mathbf{1}^\top (\mathbf{p} - \mathbf{y}_B)$$

$B = N$ is batch (Book 3), $B = 1$ is SGD ($\mathbf{X}_B$ is one row and the product is just $(p - y) \mathbf{x}$), $B = 16$ or 32 is a mini-batch. The code in Chapter 7 is this line with $B = 1$ for the tumours and $B = 16$ for the flowers.



Three ways, same 20 epochs, same 455 patients. Batch takes 20 steps in total and is still descending. Mini-batches of 16 take 29 steps per epoch. SGD takes 455 per epoch and reaches the lowest loss first. More steps per epoch means more learning per epoch — as long as the noise stays under control.

STOP AND THINK

You have 10 million labelled photos. Batch gradient descent needs one full pass through all 10 million to move the knobs once. What does SGD have done by the time batch has taken its first step?

OUR ANSWER

Ten million steps. Long before batch has finished computing its first exact gradient, SGD has walked most of the way down the hill using rough ones. For big data, the exactness of batch is a luxury nobody can afford — and, it turns out, nobody needs. This is why Book 1 said “one tiny nudge per example, a million times”.

CAREFUL — THE NOISE CUTS BOTH WAYS

Noise is a friend in a bumpy landscape (Book 5 will have those): a jittery walker can rattle out of a small dip that would trap a careful one. But noise with a large learning rate is chaos — the knobs jump around the bottom and never settle. Two standard fixes: use a smaller η than batch would, and *shrink η as training goes on* (big steps early, tiny steps late). And always plot the loss per epoch, not per step, or the jaggedness will fool you into thinking nothing is working.

WORDS TO KNOW

Stochastic gradient descent (SGD) — one example per step. **Mini-batch** — a small group of examples per step; the standard choice. **Batch size** — how many examples per step. **Epoch** — one full pass through the training data, whatever the batch size. **Shuffle** — reorder the examples randomly before every epoch. **Learning-rate schedule** — shrinking η during training.

5

The real thing: 569 tumours

One sigmoid neuron, thirty inputs, one patient at a time. Then the questions a doctor would ask about the result.

✧ SPARK — WHERE THE DATA COMES FROM

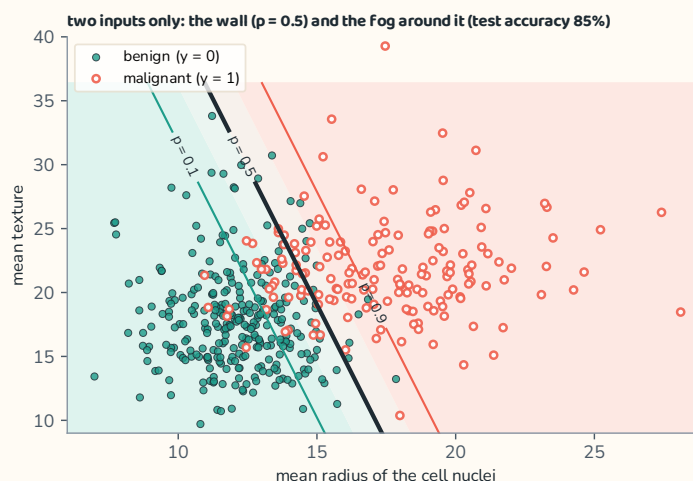
In the early 1990s, Dr William Wolberg at the University of Wisconsin was diagnosing breast lumps from fine-needle samples. Working with computer scientists Nick Street and Olvi Mangasarian, he had the cell nuclei in each microscope image outlined on a screen and measured automatically: radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension. For each image the *mean*, the *worst* (largest) and the *error* (variation) of each measurement were recorded — $3 \times 10 = 30$ numbers per patient. The **Breast Cancer Wisconsin (Diagnostic)** dataset has 569 such patients: 212 malignant, 357 benign. It has been a standard test for classifiers ever since.

Set-up, exactly as in Book 3

Label: $y = 1$ for malignant, 0 for benign. Shuffle and hide 20%: **455** patients for training, **114** hidden for testing. Standardise every input with the training mean and spread — the thirty measurements live on wildly different rulers (area in the hundreds, smoothness around 0.1). Knobs start at zero. Then SGD with $\eta = 0.05$ for 20 epochs: 455 tiny steps per epoch, 9,100 steps in all.

First, a picture with two inputs

Thirty inputs cannot be drawn, so let's look at a neuron that uses only two — mean radius and mean texture — to see what the sigmoid does to the wall.



The wall and the fog. The black line is $p = 0.5$: the perceptron's wall. Around it the sigmoid paints a fog: teal where the neuron is confident of "benign", coral where it is confident of "malignant", pale in between. With just these two measurements the neuron gets 85% of the hidden patients right — and it is honest about which ones it is unsure of.

Now all thirty

after epoch	1	2	5	10	20
training loss (average surprise)	0.074	0.060	0.050	0.045	0.040
training accuracy	98.5%	99.1%	98.9%	99.3%	99.3%

After a single epoch — 455 spoonfuls — the neuron already gets 98.5% of the training patients right. The loss keeps falling for twenty epochs while the accuracy barely moves: the neuron is not finding new right answers, it is becoming *more sure* of the ones it has. Loss and accuracy measure different things; watch both.

The honest score: 114 hidden patients

	predicted benign	predicted malignant
truly benign (67)	67	0
truly malignant (47)	3	44

accuracy = $(67 + 44) / 114 = 97.4\%$ · test loss 0.104

This table — the **confusion matrix** — says more than the accuracy. No benign tumour was called malignant (no false alarms). Three malignant tumours were called benign (three misses). In medicine those two kinds of mistake are not equal: a false alarm means an anxious week and a biopsy; a miss can cost a life. Accuracy hides that difference; the table shows it.

What the knobs say

Because the inputs are standardised, the knobs can be compared. The five largest are all positive and all point the same way: *radius error* (+1.86), *worst texture* (+1.46), *worst concavity* (+1.37), *mean concave points* (+1.33), *area error* (+1.33). In words: nuclei that are unusually large, irregular in texture, and jagged at the edges — and that vary a lot from cell to cell — push p towards malignant. A pathologist would nod. The neuron has recovered, from numbers alone, the visual signs doctors are taught to look for.

The threshold is a choice, not a law

The three missed cancers got $p = 0.07, 0.02$ and 0.04 — the neuron was fairly confident they were benign. So lowering the threshold a little does not help; lowering it a lot does:

call it malignant if $p \geq$	0.5	0.1	0.01	0.001
cancers missed	3	3	0	0
false alarms	0	2	7	19

At a threshold of 0.01 — “call it malignant unless you are 99% sure it is benign” — no cancer is missed and seven benign patients get an unnecessary biopsy. A screening programme might well take that trade. The neuron does not decide this; people do. The sigmoid’s gift is that it lets them.

A new patient walks in

Inference is the cheap part, as always: standardise the 30 measurements with the *training* mean and spread, compute $z = \mathbf{w} \cdot \mathbf{x} + b$, squash it, read $p = \sigma(z)$. For hidden patient number 1 the neuron says $p = 0.000$ — “benign, and I am very sure” — and the truth is benign. The doctor sees the probability, not just the verdict, and reads the confidence alongside her own judgement.

⚠ CAREFUL — 97% IS NOT “SOLVED”

Three in 114 is not a number you would accept from a clinic. This dataset is famously easy; the neuron is still a straight wall in 30 dimensions; and a real screening tool would need far more patients, more careful validation, and a human in the loop. What you have built is a genuine, honest classifier — not a medical device. Keep the difference clear, always.

📖 WORDS TO KNOW

Accuracy — fraction of right verdicts. **Confusion matrix** — the 2×2 table of right and wrong verdicts by kind. **False alarm (false positive)** — benign called malignant. **Miss (false negative)** — malignant called benign. **Threshold** — the p above which we say yes; a human choice that trades misses for false alarms.

6

Which of many? Softmax and the shared vote

Three species, not two. One neuron cannot say “which of three”, but three neurons can — if they agree to share a single vote of size 1.

STORY

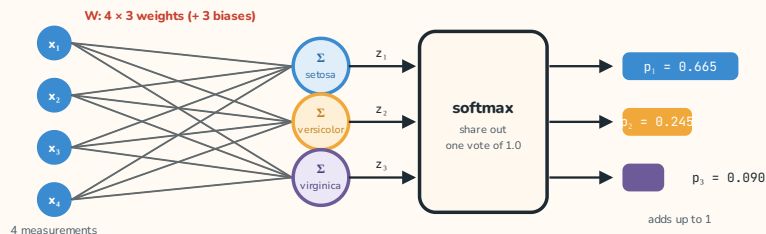
Asha's friend Rhea studies botany and cannot tell three kinds of iris apart from photographs. She has a book of 150 pressed flowers, fifty of each species — *setosa*, *versicolor*, *virginica* — with four measurements for each: the length and width of the sepal and of the petal. “Yes or no won't do,” she says. “I need *which one*.”

SPARK — THE MOST FAMOUS DATASET IN STATISTICS

The flowers are real. The botanist Edgar Anderson measured them in the Gaspé Peninsula of Canada, all on the same day, from the same pasture, with the same ruler. In 1936 the statistician Ronald Fisher used the 150 rows to invent a method for telling groups apart, and the *Iris* dataset has been the first dataset almost every student meets ever since. Ninety years on, it is still the cleanest way to learn “which of many”.

Three neurons, side by side

Give each species its own neuron. Each takes the same four inputs and produces its own **score**: z_1 for *setosa*, z_2 for *versicolor*, z_3 for *virginica*. Each score is a weighted sum, as always — so there are now $4 \times 3 = 12$ weights, best written as a grid $\mathbf{W}$ with one column per species, and 3 biases.



Three neurons and a softmax. Every neuron sees all four inputs and produces a score. Softmax converts the three scores into three probabilities that add to 1. The species with the biggest share is the verdict.

From scores to a shared vote: softmax

Scores can be any numbers — negative, huge, whatever. To turn them into probabilities we need three things: all positive, adding to 1, and bigger score means bigger probability. Softmax does exactly that:

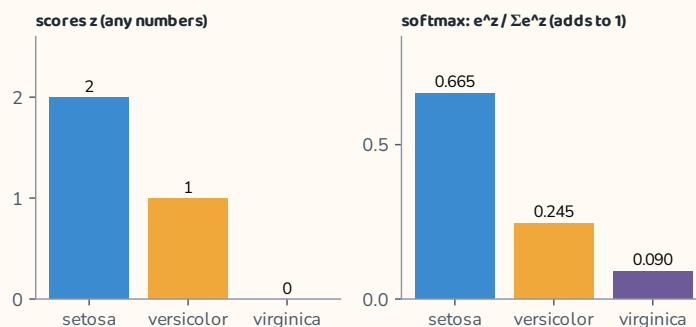
$$p_k = \frac{e^{z_k}}{e^{z_1} + e^{z_2} + e^{z_3}} \quad k = 1, 2, 3$$

raise e to each score, then divide by the total so the shares add to 1

By hand, scores (2, 1, 0):

$$e^2 = 7.389, \quad e^1 = 2.718, \quad e^0 = 1, \quad S = 7.389 + 2.718 + 1 = 11.107, \quad \mathbf{p} = \frac{(7.389, 2.718, 1)}{11.107} = (0.665, 0.245, 0.090)$$

Setosa has the biggest share; the verdict is setosa, with 66% confidence.



Softmax in one picture. Left: three scores. Right: their shares. The exponential does two jobs: it makes everything positive, and it stretches the gaps — a lead of 1 point in score becomes a lead of 2.7x in probability. Add 5 to all three scores and nothing changes; only differences matter.

THINK OF IT LIKE... A JURY THAT MUST SPLIT ONE VOTE

Three advocates each argue for their species and hand in a score. The jury has exactly one vote to share. Softmax is the rule for splitting it: a stronger argument gets a bigger slice, every advocate gets *some* slice, and the slices always total one. With only two advocates, this rule turns out to be the sigmoid from Chapter 1 exactly: $p_1 = \sigma(z_1 - z_2)$. Softmax is the sigmoid for more than two.

The truth as a vote, and surprise again

Write each label the same way the model speaks — as a split of the vote that is all-in on one species. Setosa is (1, 0, 0), versicolor (0, 1, 0), virginica (0, 0, 1). This is called **one-hot** encoding. Then the loss is the one you already know: the **surprise at the true class**:

$$L = -[y_1 \log p_1 + y_2 \log p_2 + y_3 \log p_3] = -\log p_{\text{true species}}$$

only the true species' term survives, because the other y 's are 0

Example: the model said (0.665, 0.245, 0.090) and the flower was actually versicolor, $\mathbf{y} = (0, 1, 0)$. Surprise = $-\log 0.245 = 1.41$ — a stiff penalty for leaning the wrong way. Averaged over the flowers, this is the **cross-entropy**; the binary version of Chapter 2 is the special case with two classes.

The slope: $p - y$, one more time

Here is the derivation for the true class c , using S for the total $e^{z_1} + e^{z_2} + e^{z_3}$. Then the same thing for a whole mini-batch, as matrices.

PLAIN MATHS — one flower, one score at a time

1 $L = -\log \frac{e^{z_c}}{S} = -(z_c - \log S)$

minus the log of the true class's share; $\log(a/b) = \log a - \log b$

2 $\frac{\partial z_c}{\partial z_k} = y_k, \quad \frac{\partial \log S}{\partial z_k} = \frac{1}{S} \cdot e^{z_k} = p_k$

nudge score z_k : the first part moves by 1 if k is the true class, 0 otherwise — exactly y_k ; the second part is the slope of $\log S$ times the slope of S

3 $\frac{\partial L}{\partial z_k} = -(y_k - p_k) = p_k - y_k$

put them together, remembering the leading minus sign

4 $\frac{\partial L}{\partial w_{jk}} = (p_k - y_k) x_j \quad \frac{\partial L}{\partial b_k} = p_k - y_k$

attach the input, exactly as for the sigmoid neuron: the weight from input x_j to neuron k , and neuron k 's bias

For our example — model (0.665, 0.245, 0.090), truth versicolor (0, 1, 0) — the score-gradient is $\mathbf{p} - \mathbf{y} = (0.665, -0.755, 0.090)$: push setosa's score down, versicolor's up by a lot, virginica's down a little. Each neuron is corrected by its own share of the blame.

↓ a mini-batch of B flowers: rows are flowers, columns are species

MATRIX FORM — every flower, every neuron, every knob at once

1 $\mathbf{Z} = \mathbf{XW} + \mathbf{b} \quad (B \times 3)$

$\mathbf{X}$ is $B \times 4$ (four measurements), $\mathbf{W}$ is 4×3 (one column of knobs per species), $\mathbf{b}$ is a row of 3 added to every row

2 $\mathbf{P} = \text{softmax}(\mathbf{Z}) \quad (B \times 3)$

softmax applied to each row: every flower's three shares add to 1

3 $L = -\frac{1}{B} \sum_{i=1}^B \sum_{k=1}^3 Y_{ik} \log P_{ik}$

$\mathbf{Y}$ is the $B \times 3$ one-hot matrix; only the true entry of each row survives

4 $\nabla_{\mathbf{W}} L = \frac{1}{B} \mathbf{X}^T (\mathbf{P} - \mathbf{Y}) \quad (4 \times 3) \quad \nabla_{\mathbf{b}} L = \frac{1}{B} \mathbf{1}^T (\mathbf{P} - \mathbf{Y})$

step 4 for the whole grid: entry (j, k) is $\frac{1}{B} \sum_i (P_{ik} - Y_{ik}) x_{ij}$ — error $\times$ input, added over the batch, for every weight at once. Same shape as $\mathbf{W}$, so it can be subtracted from it.

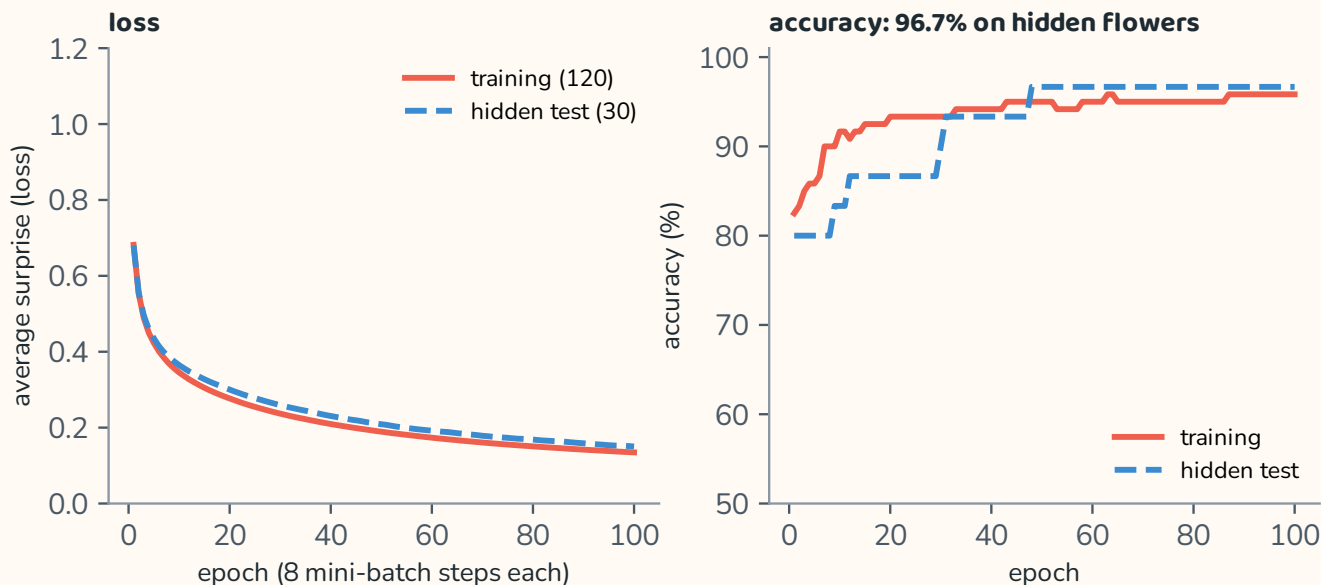
One flower, as matrices ($B = 1$): the gradient is an outer product — the error row scaled by each input.

$$\nabla_{\mathbf{W}} L = \mathbf{X}^T (\mathbf{P} - \mathbf{Y}) = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \begin{bmatrix} 0.665 & -0.755 & 0.090 \end{bmatrix} = \begin{bmatrix} 0.665 x_1 & -0.755 x_1 & 0.090 x_1 \\ 0.665 x_2 & -0.755 x_2 & 0.090 x_2 \\ 0.665 x_3 & -0.755 x_3 & 0.090 x_3 \\ 0.665 x_4 & -0.755 x_4 & 0.090 x_4 \end{bmatrix}$$

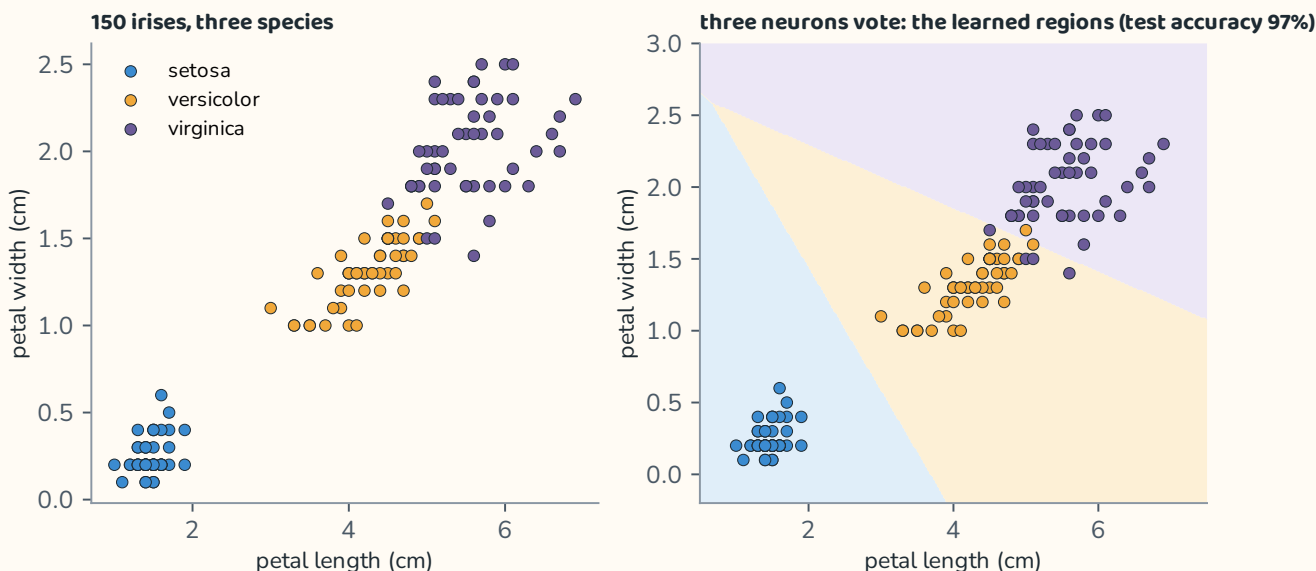
column k is neuron k 's blame times the inputs — the sigmoid neuron's $(p - y) \mathbf{x}$, three times side by side. In the notebook: `dW = X[bi].T @ (P - Y[bi]) / len(bi)`.

Train with mini-batches, then look

Split 120 / 30, standardise, knobs at zero, mini-batches of 16 (so 8 steps per epoch), $\eta = 0.1$, 100 epochs.



Mini-batch training on Iris. The loss falls smoothly from $\log 3 = 1.10$ (a three-way coin toss) to 0.13. Accuracy climbs to **96.7%** on the 30 hidden flowers — 29 right, one virginica called versicolor.



What three neurons draw. Left: the flowers, on the two petal measurements. Right: the regions where each neuron wins the vote, from a model trained on those two inputs alone. Three straight walls meeting at a point — the most a row of linear neurons can do. Setosa is easy; versicolor and virginica overlap where the walls meet.

true \ predicted	setosa	versicolor	virginica
setosa (11)	11	0	0
versicolor (10)	0	10	0
virginica (9)	0	1	8

Reading W. Take the row for petal length: $(-1.95, +0.06, +1.90)$. Long petals push the setosa score down hard and the virginica score up hard, leaving versicolor in the middle. The petal-width row does the same, more strongly for virginica $(+2.70)$. The sepal rows are smaller: sepals matter less. Three neurons have learned, from 120 flowers, what botanists take a semester to teach.

A new flower. Measurements (5.1, 3.5, 1.4, 0.2) — small petals. Standardise with the training mean and spread, multiply by W , add b , softmax: **(0.992, 0.008, 0.000)**. Setosa, and sure of it. A flower with measurements (6.5, 3.0, 5.2, 2.0) gives (0.000, 0.118, 0.882): virginica, with a nod to versicolor. The vote carries the doubt, as it should.

WORDS TO KNOW

Score (logit) z_k — the k -th neuron's raw total. **Softmax** — $e^{z_k} / \sum e^{z_j}$; turns scores into shares that sum to 1. **One-hot** — the label written as a vote all-in on one class. **Cross-entropy** — average surprise at the true class. **Weight matrix W** — one column of knobs per class. **Decision region** — the part of input space where one class wins.

7

The demo: two classifiers in fifty lines

The sigmoid neuron with SGD, and the softmax layer with mini-batches. Every number in Chapters 3, 5 and 6 comes from these cells.

The companion notebook `Book4_Classification_From_Scratch.ipynb` needs no downloads: both datasets ship with scikit-learn, which we use *only* to load them. The learning is NumPy, written by hand. Here are the two classes, with the four learning-loop steps marked.

1 • Sigmoid + surprise + SGD

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def surprise(p, y, eps=1e-12):
    # binary cross-entropy = average surprise
    p = np.clip(p, eps, 1 - eps) # never take log of exactly 0
    return -np.mean(y*np.log(p) + (1-y)*np.log(1-p))

class SigmoidNeuron:
    def __init__(self, n_inputs, lr=0.05):
        self.w = np.zeros(n_inputs); self.b = 0.0; self.lr = lr

    def prob(self, X):
        return sigmoid(X @ self.w + self.b) # 'how sure is it yes', one per row

    def predict(self, X):
        return (self.prob(X) ≥ 0.5).astype(int) # the wall is still at z = 0

    def fit_sgd(self, X, y, epochs=20, seed=0):
        rng = np.random.default_rng(seed)
        for _ in range(epochs):
            for i in rng.permutation(len(y)):
                # shuffle, then ONE example at a time
                p = sigmoid(X[i] @ self.w + self.b) # 1 · guess
                err = p - y[i] # 2 · measure: this IS dL/dz (Chapter 3)
                self.w = self.w - self.lr * err * X[i] # 3 · nudge: dL/dw = (p - y) x
                self.b = self.b - self.lr * err # dL/db = (p - y)
            # 4 · repeat (the loops above)
        return self
```

```
training accuracy 99.3%   test accuracy 97.4%
training loss 0.040   test loss 0.104
malignant caught (TP) 44   benign cleared (TN) 67   false alarms (FP) 0   missed cancers (FN) 3
```

2 • Softmax + cross-entropy + mini-batches

```
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True) # subtract each row's max first: same shares, no overflow
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)

def one_hot(y, k):
    Y = np.zeros((len(y), k)); Y[np.arange(len(y)), y] = 1; return Y

class SoftmaxLayer:
    def __init__(self, n_inputs, n_classes, lr=0.1):
        self.W = np.zeros((n_inputs, n_classes)); self.b = np.zeros(n_classes); self.lr = lr

    def prob(self, X):
        return softmax(X @ self.W + self.b) # scores for every class, then shares

    def predict(self, X):
        return np.argmax(self.prob(X), axis=1) # the biggest share wins

    def fit_minibatch(self, X, y, epochs=100, batch=16, seed=0):
        Y = one_hot(y, self.W.shape[1]); rng = np.random.default_rng(seed)
        for _ in range(epochs):
            idx = rng.permutation(len(y))
            for s in range(0, len(y), batch):
                bi = idx[s:s+batch] # the next 16 flowers
                P = self.prob(X[bi]) # 1 · guess
                D = (P - Y[bi]) / len(bi) # 2 · measure: dL/dZ = P - Y, averaged
                self.W = self.W - self.lr * (X[bi].T @ D) # 3 · nudge every weight at once
                self.b = self.b - self.lr * D.sum(axis=0)
            # 4 · repeat (the loops above)
        return self
```

```

training accuracy 95.8%   test accuracy 96.7%
[5.1 3.5 1.4 0.2] → {'setosa': 0.992, 'versicolor': 0.008, 'virginica': 0.0} → verdict: setosa
[6.5 3.  5.2 2. ] → {'setosa': 0.0, 'versicolor': 0.118, 'virginica': 0.882} → verdict: virginica

```

Notice how little changed: `sigmoid` became `softmax`, a vector of knobs became a grid, $p - y$ became $P - Y$. Every classifier that ends in a `softmax` — including the ones that name objects in photos — has this exact shape at its output.

3 • What else is in the notebook

Section	What it does	Book chapter
1	The one-patient toy: p , surprise, gradient $(p - y) \cdot x$, one step, new p .	Chapter 3
2 – 3	The sigmoid class; the 569 tumours; split, standardise, train with SGD; loss and accuracy curves; the confusion matrix; the six biggest knobs; the threshold experiment.	Chapters 4, 5
4	Batch vs mini-batch vs SGD on the same data and epochs.	Chapter 4
5	Softmax and one-hot by hand; the softmax layer; Iris with mini-batches; W and b ; the confusion matrix; two new flowers.	Chapter 6
6	Six things to try: the threshold curve; SGD learning rates; two inputs only; softmax with two classes (it is the sigmoid); Iris without scaling; a dataset no straight wall can solve.	everything

The story in six sentences

1. Replace the perceptron's cliff with the **sigmoid** squash, $p = 1/(1 + e^{-z})$, and one neuron says "how sure it is yes" — a number between 0 and 1, with the old wall still at $p = 0.5$.
2. Score it by **surprise**, $-\log$ of the probability given to the truth (the binary **cross-entropy**): confident and right costs nothing, confident and wrong costs without limit, and the loss never goes flat.
3. The chain from a knob to the loss has three links — x , then $p(1 - p)$, then $-1/p$ — and they multiply to the cleanest gradient in the course: $\partial L / \partial z = p - y$, so $\partial L / \partial w = (p - y)x$; for a batch, $\nabla_{\mathbf{w}} L = \frac{1}{N} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$.
4. **Stochastic gradient descent** steps after every single example; **mini-batches** step after a handful; both trade exactness for many more steps per epoch, and the noise averages out.
5. On 569 tumours, one sigmoid neuron reaches 97.4% on hidden patients; the **confusion matrix** and the **threshold** tell the story that accuracy hides.
6. For "which of many", one neuron per class produces a score, **softmax** shares out a single vote, **one-hot** labels write the truth the same way, and the gradient is again $\mathbf{P} - \mathbf{Y}$ (so $\nabla_{\mathbf{w}} L = \frac{1}{B} \mathbf{X}^T (\mathbf{P} - \mathbf{Y})$) — 96.7% on the irises.

Explain it to a friend

Cover the right column. Answer out loud, in your own words. Then compare.

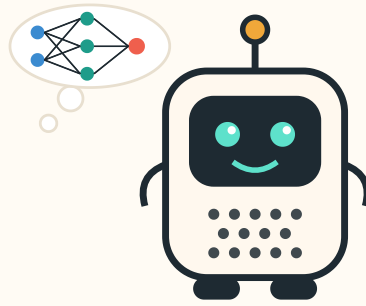
Question	A good short answer
What does the sigmoid change about the perceptron?	Only the last step: a smooth squash instead of a cliff. The output becomes a confidence; the decision wall is unchanged.
Why is $\sigma(0) = 0.5$ important?	Because " $p \geq 0.5$ " is the same as " $z \geq 0$ " — the sigmoid neuron and the perceptron draw the same line.
What is surprise?	$-\log$ of the probability the model gave to what actually happened. Zero if it was sure and right; unbounded if it was sure and wrong.
Why not squared error with a sigmoid?	It goes flat when the model is confidently wrong, so the gradient dies exactly when a big correction is needed.
What is the slope of the sigmoid?	$p(1 - p)$: steep at $p = 0.5$, flat near 0 and 1 — readable from the output itself.
Why does the gradient collapse to $p - y$?	The sigmoid's $p(1 - p)$ cancels against the surprise's $1/p$ and $1/(1 - p)$. The two were made for each other.
What is SGD, in one line?	Shuffle; for each example, nudge the knobs by $\eta \cdot (p - y) \cdot x$; repeat for several epochs.
Why mini-batches?	Less noise than one example, far cheaper than all of them, and GPUs process a batch in one go.
What does softmax do?	Turns any scores into positive shares that add to 1, keeping the order and stretching the gaps.
Why one-hot?	So the truth is written as a vote all-in on one class, and cross-entropy becomes $-\log$ of the true class's share.

❖ QUESTIONS WORTH CARRYING (NO ANSWERS HERE — ON PURPOSE)

- 1 · Cross-entropy punishes a 0.01 guess that turns out true by 4.6, and a 0.5 guess by 0.69. A model can lower its loss by never being confident. Why doesn't training collapse into permanent hedging?
- 2 · The perceptron rule was "nudge on a mistake by $\pm 1 \times \text{input}$ ". SGD on the sigmoid neuron is "nudge every time by $(p - y) \times \text{input}$ ". Write down the one situation where the two rules give the same nudge.
- 3 · Softmax's regions on Iris were three straight walls meeting at a point. Sketch a three-class dataset where that is hopeless. What would fix it — more neurons in the same row, or a second row?
- 4 · One-hot labels say the truth is 100% one class. Real labels are sometimes uncertain (two doctors disagree). What could you feed the loss instead of a one-hot vector, and what would the gradient $\mathbf{P} - \mathbf{Y}$ do with it?

Words to know — the whole book

Sigmoid $\sigma(z)$ $1/(1 + e^{-z})$: squashes any total into (0, 1).	Activation function The last step of a neuron: step, identity, sigmoid, softmax...	Surprise $-\log p_{\text{truth}}$: the cost of being wrong, unbounded.
Binary cross-entropy $-[y \log p + (1 - y) \log(1 - p)]$, averaged.	Cross-entropy Average $-\log$ of the true class's share; the multi-class version.	$\sigma'(z) = p(1 - p)$ The slope of the sigmoid, in terms of its own output.
$p - y$ The gradient of the loss with respect to the score; error $\times$ input follows.	SGD One example per step; noisy, cheap, many steps per epoch.	Mini-batch A handful of examples per step; the practical default.
Shuffle Randomise the order every epoch.	Accuracy Fraction of correct verdicts on a set.	Confusion matrix Right and wrong verdicts, broken down by kind.
False alarm / miss Benign called malignant / malignant called benign.	Threshold The p above which we say yes; a human choice.	Score (logit) A class neuron's raw total before softmax.
Softmax $e^{z_k} / \sum e^{z_j}$: scores $\rightarrow$ shares that sum to 1.	One-hot A label written as (0, ..., 1, ..., 0).	Weight matrix W One column of knobs per class; gradient $X^T(P - Y)/B$.
Decision region Where one class wins the vote; straight walls for linear neurons.	Logistic regression The statistics name for the sigmoid neuron.	Calibration Whether the model's 0.9s are right nine times in ten.



End of Book 4

Nova can now say “how much”, “how sure”, and “which one”. Every wall it draws is still straight.

NEXT TIME — BOOK 5: A REAL BRAIN

Everything is on the table. A smooth neuron (sigmoid) whose slope we can read off its output. A loss (surprise) whose gradient at the output is simply $p - y$. A walk (gradient descent, in batches or one at a time). And the chain rule: multiply the links.

In Book 5 we put neurons in **layers** — a hidden row of them feeding an output row — and the chain grows longer. We run the network forwards with matrices, and then we send the error *backwards* through the same wires, giving every hidden neuron its share of the blame. That is **backpropagation**. It solves the puzzle from Book 2: XOR falls in a few hundred steps. It bends the walls. And, at the very end, it is what lets Nova hear its name.

Deep Learning, Told Simply · a five-book story for the Deep Learning (DNN) course · Prof. Saurabh